

CSC3173: Artificial Intelligence (2024/25)

Programming Assignment 01

Customer Churn Prediction: XGBoost vs MLP

Registration Number: S/21/513

1. Introduction

Customer churn prediction is a binary-classification task used to identify customers who are likely to discontinue a service. This assignment compares **XGBoost** and a **Multilayer Perceptron (MLP)** using the IBM Telco Customer Churn dataset. Performance is evaluated using **Accuracy, Precision, Recall, and F1-score**. Since the dataset is imbalanced, Recall and F1-score are particularly useful for evaluating churn detection.

2. Dataset and Preprocessing

The dataset contains **7,043 customer records** and **33 original columns**. The target variable is **Churn Value**, where 0 represents no churn and 1 represents churn. The dataset contains 5,174 non-churn customers (73.46%) and 1,869 churn customers (26.54%).

Columns that could cause target leakage or provide unnecessary identifier/location information were removed. These included **CustomerID**, location-related fields, **Churn Label**, **Churn Score**, **CLTV**, and **Churn Reason**. The **Total Charges** column was converted to numeric form, and 11 missing values were replaced using the median.

The final dataset contained **19 predictor variables**: 16 categorical and 3 numerical. Categorical variables were one-hot encoded, while numerical variables were standardized using **StandardScaler**. An 80:20 stratified train/test split was used with **random_state=42**. After preprocessing, the input contained **46 features**.

3. Implementation Environment

The experiment was implemented using Python 3.12 in a uv-managed virtual environment and executed in Jupyter-Lab.

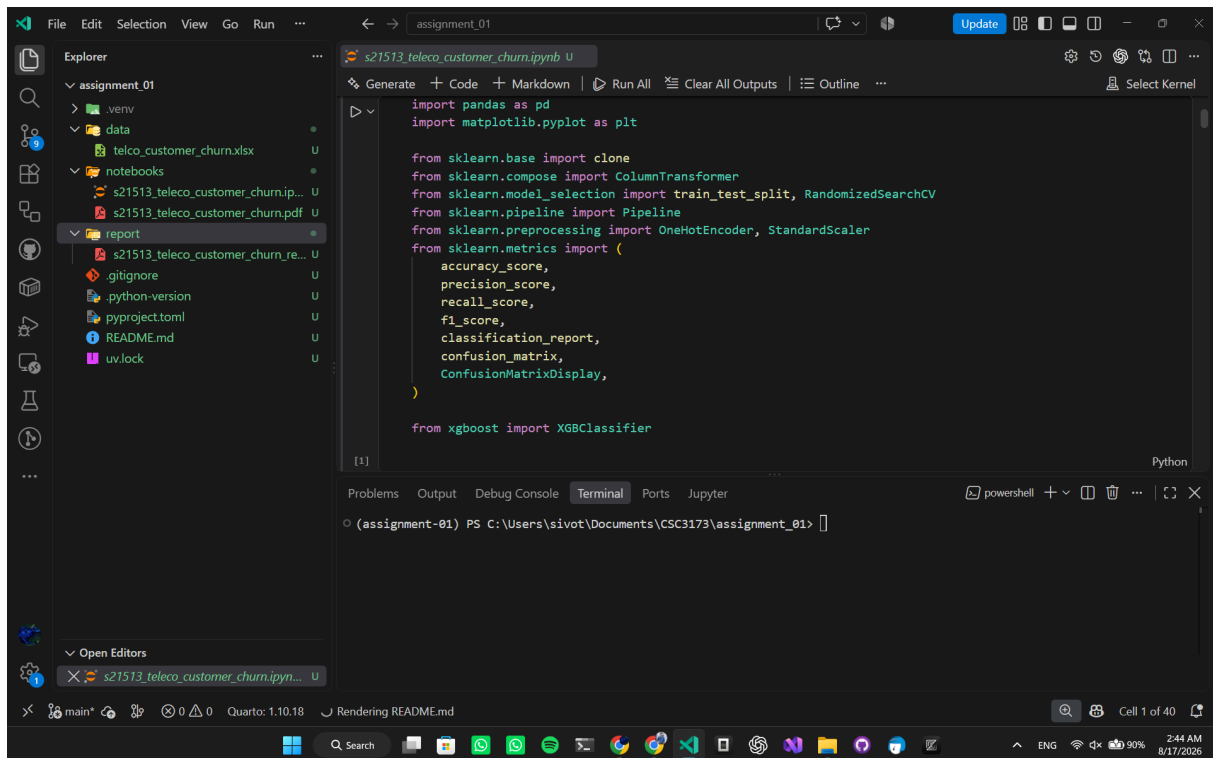


Figure 1: Development environment used to execute the assignment.

4. XGBoost Classifier

4.1. Baseline XGBoost

A baseline `XGBClassifier` was trained using the preprocessed training data.

Table 1: Baseline XGBoost performance

Metric	Score
Accuracy	0.7892
Precision	0.6207
Recall	0.5294
F1-score	0.5714

4.2. Hyperparameter Optimization

Hyperparameter optimization was performed using `RandomizedSearchCV`. A 5-fold cross-validation strategy was applied to the training set, and F1-score was used as the optimization criterion. Thirty parameter combinations were evaluated, resulting in 150 model fits.

Table 2: Best XGBoost hyperparameters

Hyperparameter	Best Value
<code>n_estimators</code>	200
<code>max_depth</code>	3
<code>learning_rate</code>	0.05
<code>subsample</code>	0.9
<code>colsample_bytree</code>	0.8
<code>min_child_weight</code>	5

The best cross-validation F1-score was **0.6142**.

Table 3: Optimized XGBoost performance

Metric	Score
Accuracy	0.8070
Precision	0.6656
Recall	0.5481
F1-score	0.6012

Hyperparameter optimization improved all four reported metrics compared with the baseline XGBoost model.

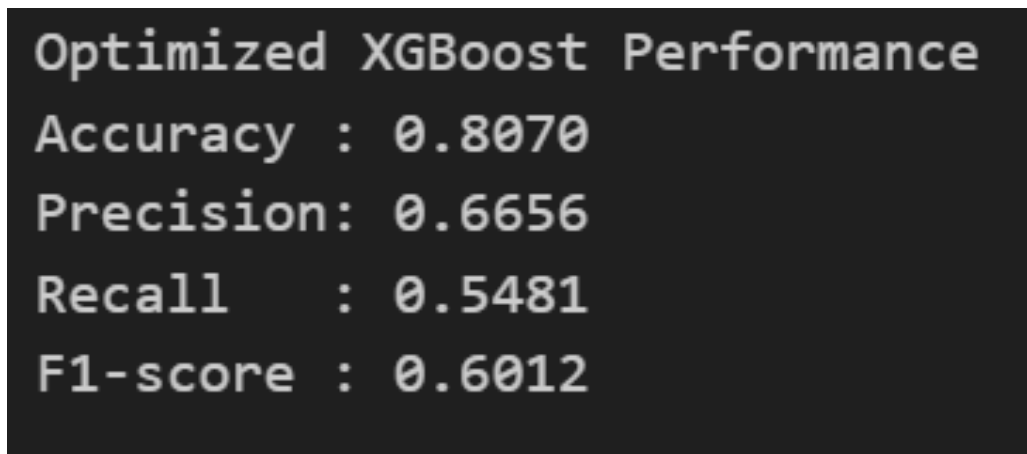


Figure 2: Sample XGBoost code and generated performance output.

5. Multilayer Perceptron (MLP)

5.1. Baseline MLP

The baseline MLP consisted of two hidden ReLU layers containing 64 and 32 neurons, followed by a single sigmoid output neuron. The model used the Adam optimizer, binary cross-entropy loss, batch size 32, and early stopping based on validation loss.

Table 4: Baseline MLP performance

Metric	Score
Accuracy	0.7828
Precision	0.5813
Recall	0.6497
F1-score	0.6136

5.2. Hyperparameter Optimization

For MLP hyperparameter selection, the original training set was divided into internal training and validation subsets. Candidate models varied hidden-layer size, learning rate, dropout rate, and batch size. Validation F1-score was used to select the best configuration.

Table 5: Best MLP hyperparameters

Hyperparameter	Best Value
First hidden layer	64 neurons
Second hidden layer	32 neurons
Learning rate	0.0005
Dropout rate	0.3
Batch size	32
Epochs	27

The best validation F1-score was **0.6484**. A fresh MLP was then trained on the full training set using the selected configuration.

Table 6: Optimized MLP performance

Metric	Score
Accuracy	0.8013
Precision	0.6655
Recall	0.5053
F1-score	0.5745

Although tuning improved MLP accuracy and precision, Recall and F1-score decreased on the final test set.

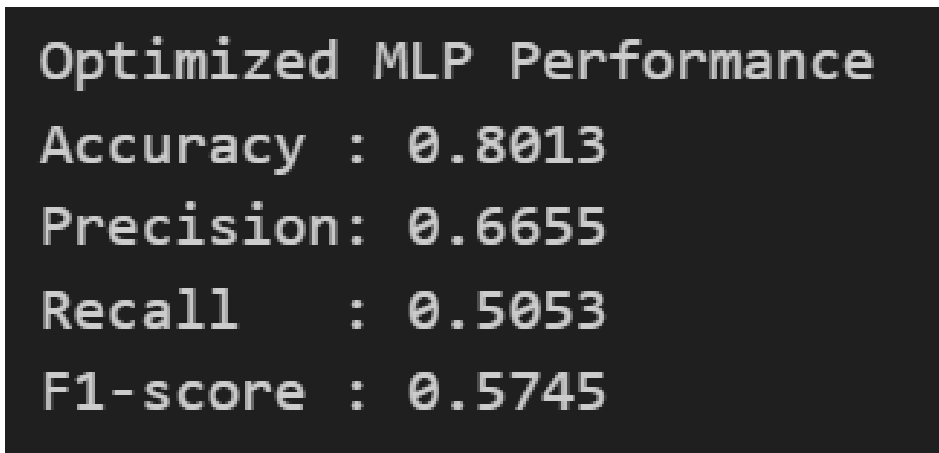


Figure 3: Sample MLP code and generated performance output.

6. Final Performance Comparison

Table 7: Performance comparison of all models

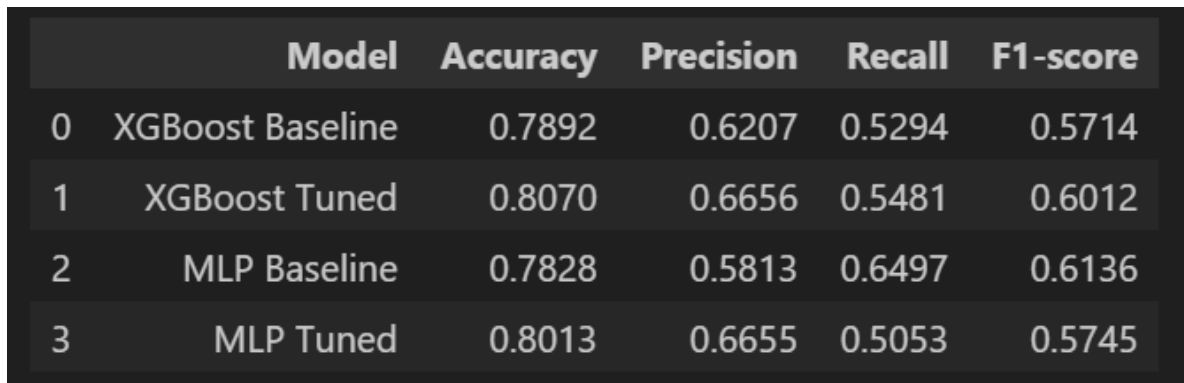
Model	Accuracy	Precision	Recall	F1-score
XGBoost Baseline	0.7892	0.6207	0.5294	0.5714
XGBoost Tuned	0.8070	0.6656	0.5481	0.6012
MLP Baseline	0.7828	0.5813	0.6497	0.6136
MLP Tuned	0.8013	0.6655	0.5053	0.5745

7. Discussion

The tuned XGBoost model achieved the highest **accuracy (0.8070)** and **precision (0.6656)**. Hyperparameter tuning increased its F1-score from 0.5714 to 0.6012.

The baseline MLP achieved the highest **recall (0.6497)** and **F1-score (0.6136)**, indicating that it identified a larger proportion of customers who actually churned. In a customer-retention application, Recall is important because a false negative represents a churn customer who was not detected.

The tuned MLP achieved relatively high accuracy and precision but lower Recall and F1-score than the baseline MLP. This result demonstrates the importance of evaluating the selected model on an untouched test set instead of relying only on validation performance.



	Model	Accuracy	Precision	Recall	F1-score
0	XGBoost Baseline	0.7892	0.6207	0.5294	0.5714
1	XGBoost Tuned	0.8070	0.6656	0.5481	0.6012
2	MLP Baseline	0.7828	0.5813	0.6497	0.6136
3	MLP Tuned	0.8013	0.6655	0.5053	0.5745

Figure 4: Confusion matrices for the tuned XGBoost and MLP models.

8. Conclusion

Both XGBoost and MLP were successfully applied to customer churn prediction. The **tuned XGBoost** model was strongest when Accuracy and Precision were prioritized, while the **baseline MLP** achieved the highest Recall and F1-score.

Therefore, tuned XGBoost is preferable when predictive correctness and precision are the main objectives, whereas the baseline MLP is preferable when identifying a larger proportion of actual churn customers is more important.

References

1. IBM Telco Customer Churn Dataset. *Kaggle*.
<https://www.kaggle.com/datasets/yeanzc/telco-customer-churn-ibm-dataset>
2. Project source code. *GitHub*.
https://github.com/sivothajan/s21513_csc3173_teleco_customer_churn